Jenkins lab 1

Part 1 – Explore IDE and create a Jenkin job

Objectives

- In this part you will install Jenkins using Docker technology.
- Investigate the Jenkins IDE.
- Create a basic Job and explore its options

Install Jenkins using Docker

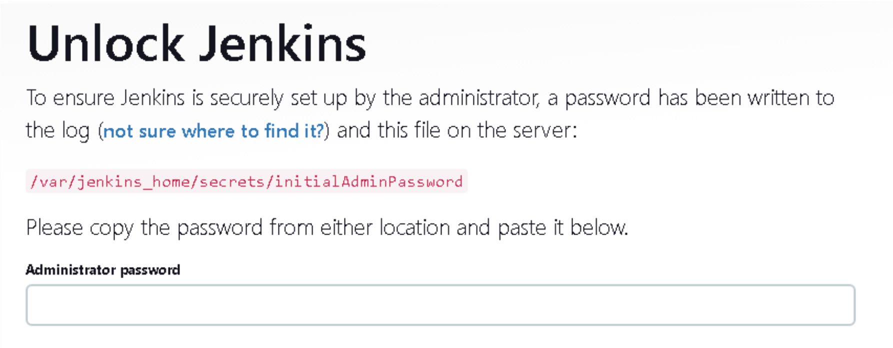
- For a simple method, type the following command

  docker run -d -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts

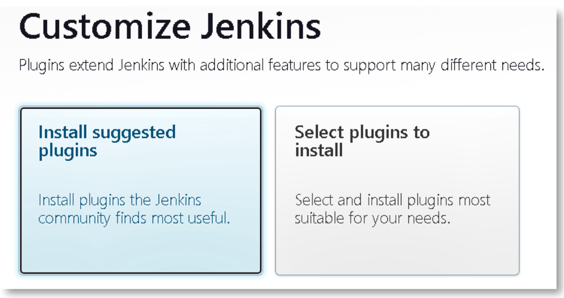
1- Get the Jenkin's docker ID (`docker ps`).
In this example we assume the ID starts with **e7b**

2- Run the following command to get the Jenkins secret password:
`docker exec -it e7b cat /var/jenkins_home/secrets/initialAdminPassword`
copy the secret password

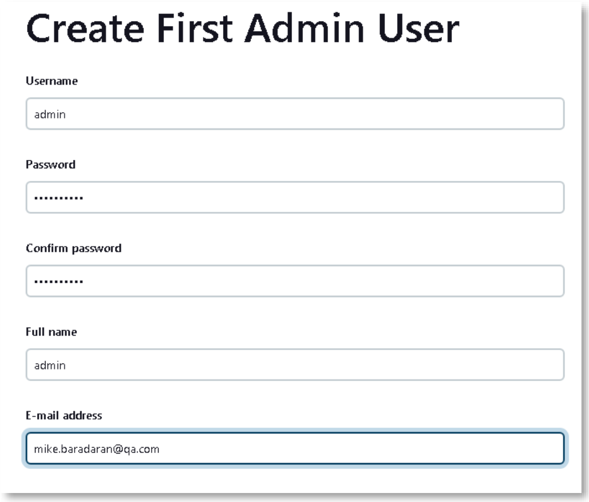
3- In a browser type: **localhost:8080**
Copy the secret Administrator password from the previous step and paste here:



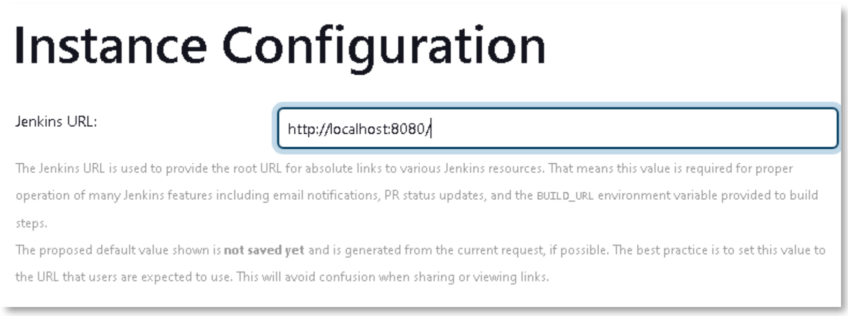
4- Install the suggested plugins

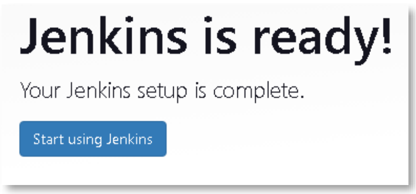


5- Create the admin user (any name and password will do. The email is not checked)

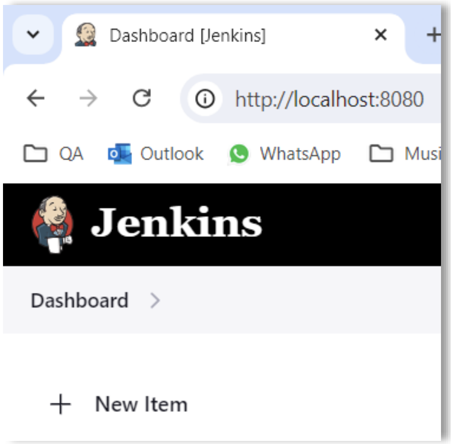


6- Follow the rest of the dialogs

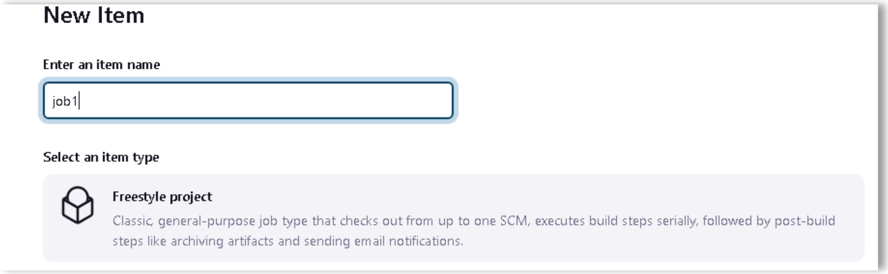




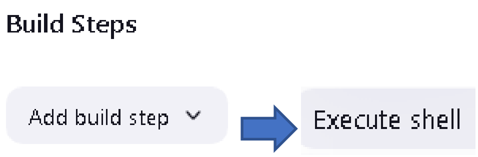
7- Create a new job by clicking [+ **New Item**]

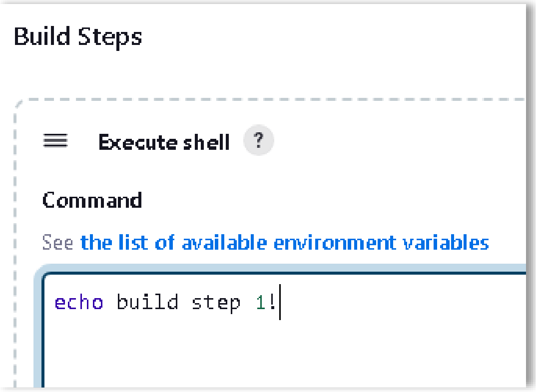


8- Choose a name for this job



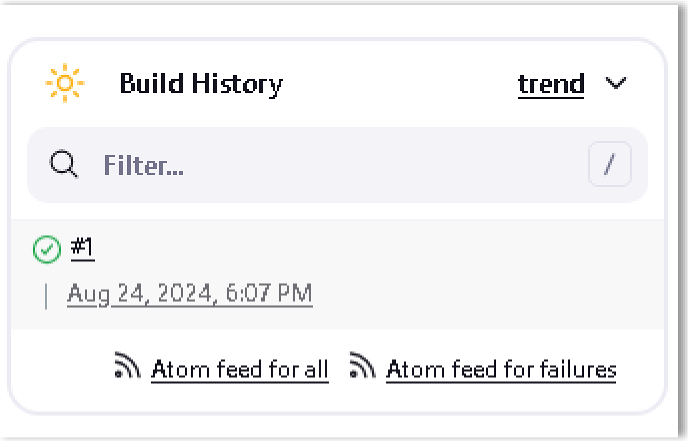
9- Scroll down and find the Buil Step section and then select these options:

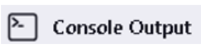


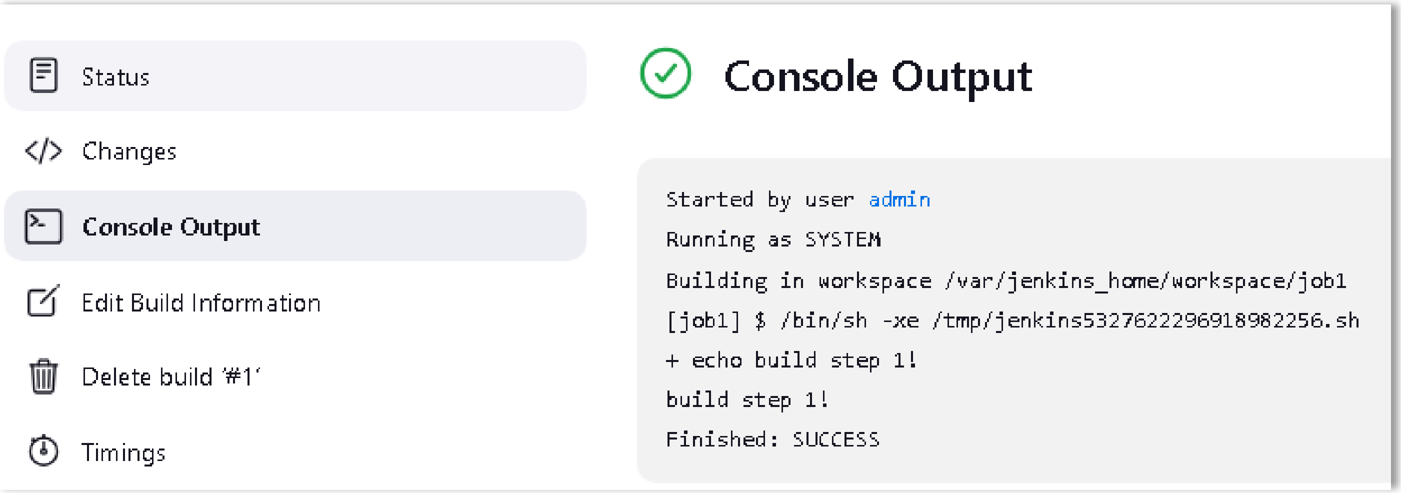


We will not deploy an application, only a simple script to get to know this tool.

10- Click **Save** and then 

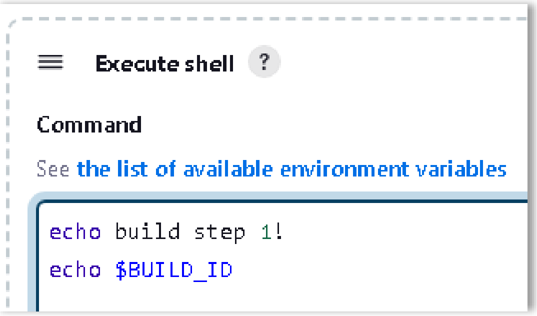


11- Click on #1 (build number) and then click 



12- Jenkins provides many environmental variables. We will have a look at a few. View these by clicking on the See the list of available environment variables link.

13- Click on **job1** title and then select the configure option

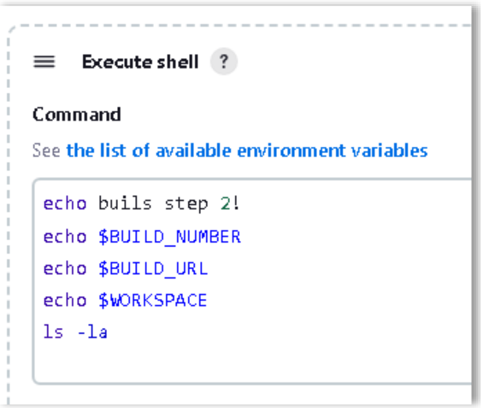


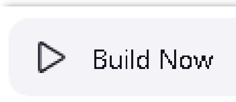
14- Click 

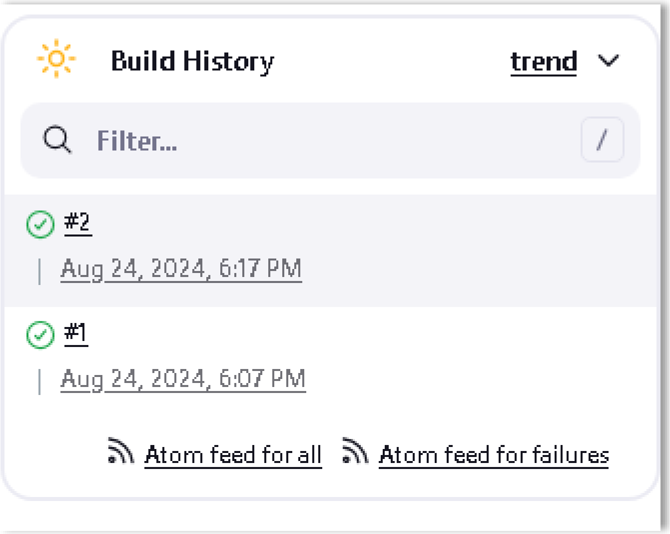
15- View the Console log and note the BUILD_ID value.

16- Modify Job1 by adding a few more lines

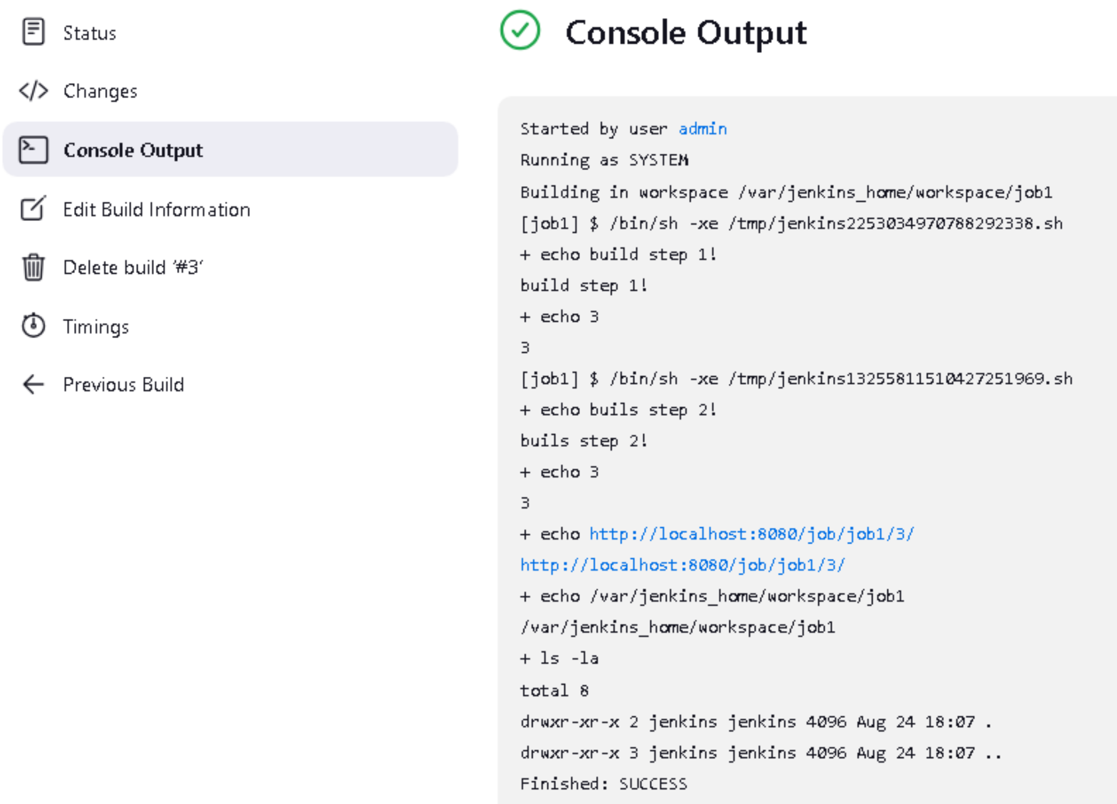
(Execute shell)



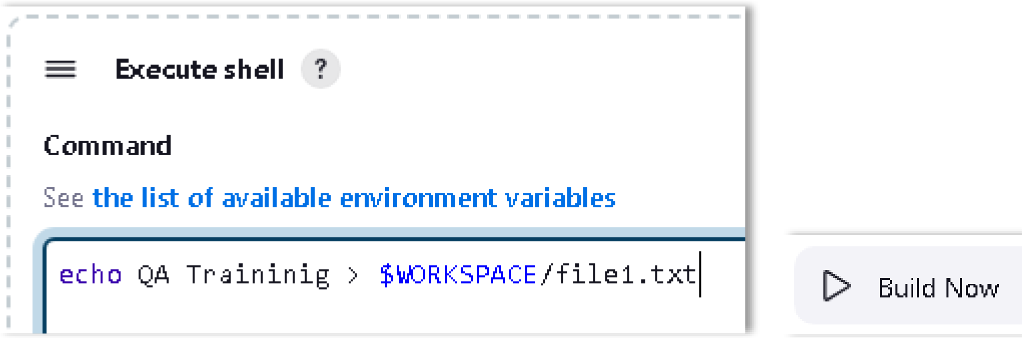




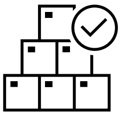
17- View the console log



18- Add another step to the build to execute a shell command to reference Jenkin's Workspace which is a directory on the Jenkins server where the files related to a specific job are stored during the build process. The following command create a file called file.txt with content of "QA Training".



19- Lokk for the Job 1’s Workspace menu and display the content of file.txt



Congratulations, you have successfully created a job using Jenkins and explored it’s various IDE options. Please continue to **Part 2** below.

Part 2- Triggers

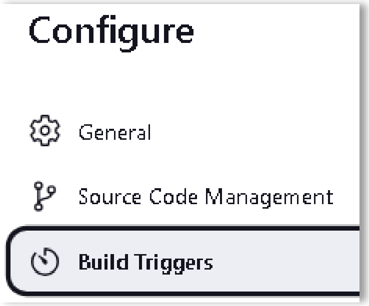
Objectives

In this part you will create a Jenkin **trigger**, and explore other forms of triggers in other labs.

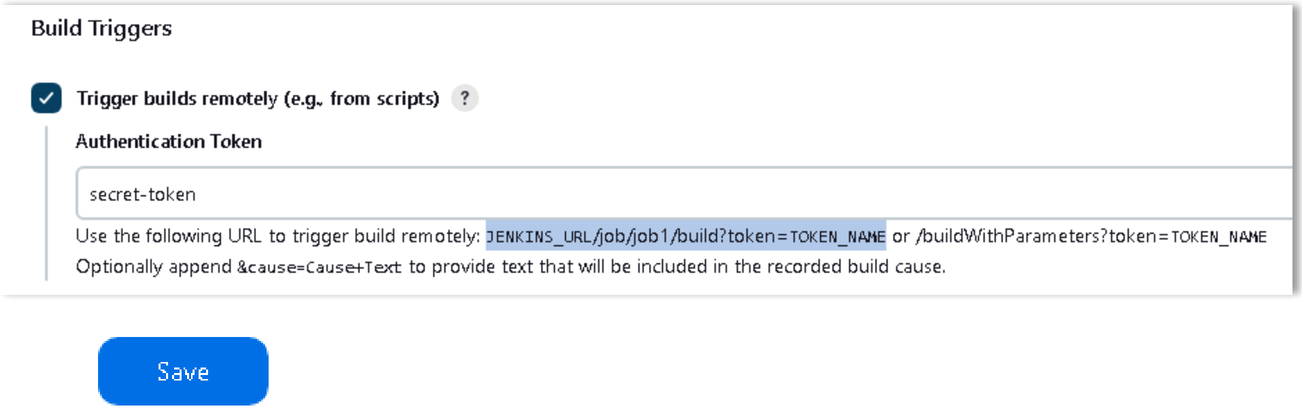
Trigger is an event or action that initiates the execution of a job or pipeline (we will explore pipelines later).

Create a Remote build trigger

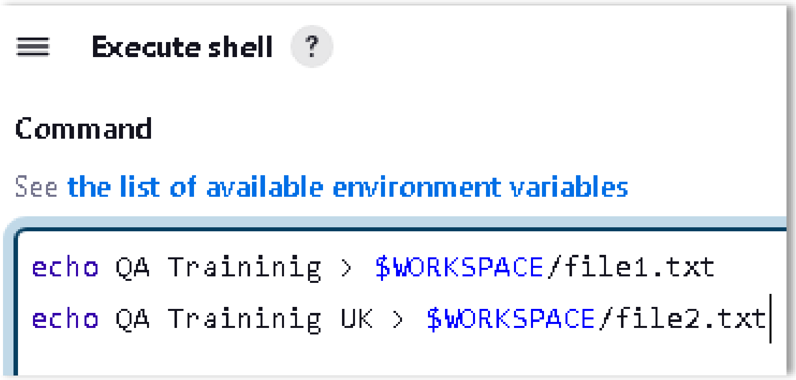
1- Configure Job1 and find the Buil Triggers section



2- Create a token for invoking and API end point for triggering a build (any secret token word will do)

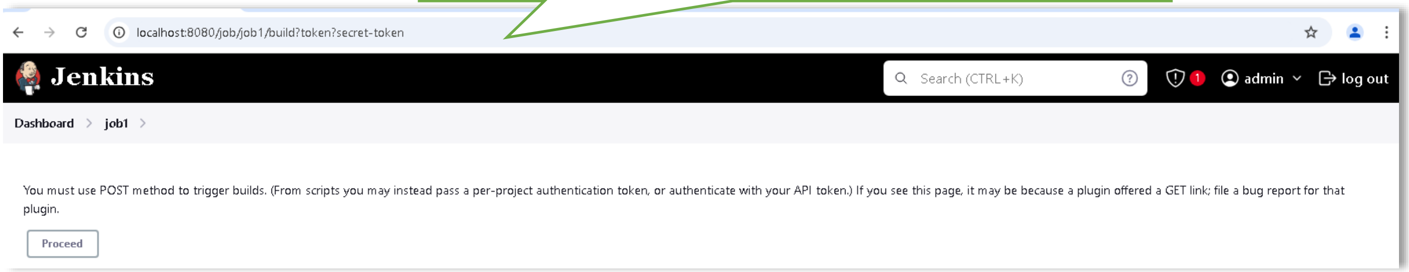


3- Make a change to the last build and then save.

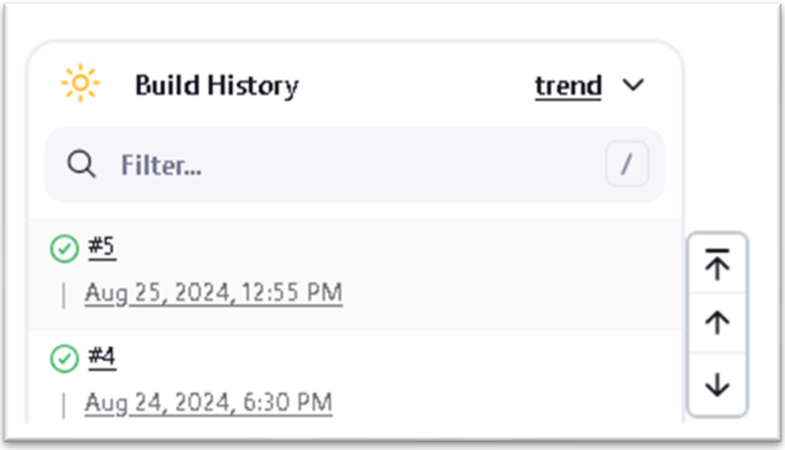


4- Type the following text in a browser and click the Proceed button to trigger the build. Please note, this will result in a blank screen and will only shows errors if there are any.

http://localhost:8080/job/job1/build?token?secret-token



5- As mentioned, the screen will go blank but a new build is triggered (please wait a moment until it does)



Congratulations, you have successfully created a Jenkins trigger. This is an important concept and you will explore other forms of Jenkin's Triggers in other labs.

Please continue to **Part 3** below.

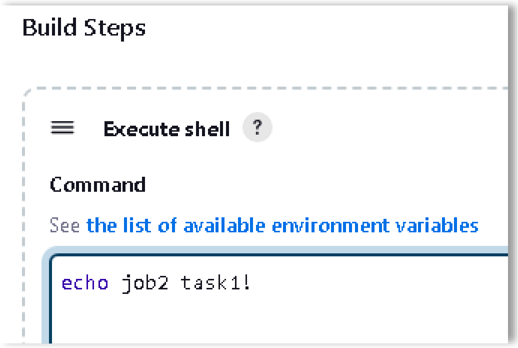
Part 3 - Chaining project builds

Objectives

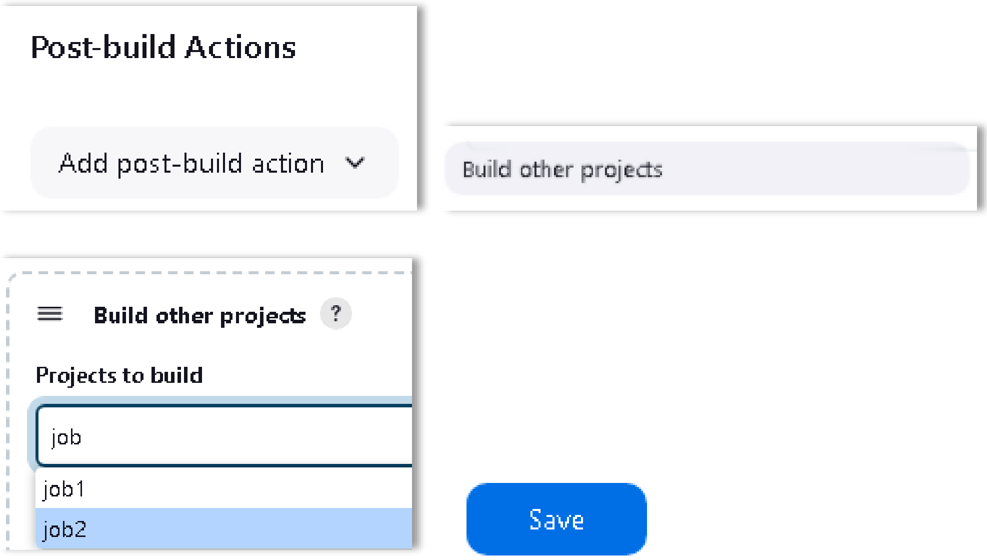
In this part you will chain 2 or more builds together.
Build chaining in Jenkins automatically triggers one or more jobs after another job completes. This enables multi-step automation pipelines, where one job's output becomes the next job's input.

1- Create a new Job called **job2**

2- Assign a build step like

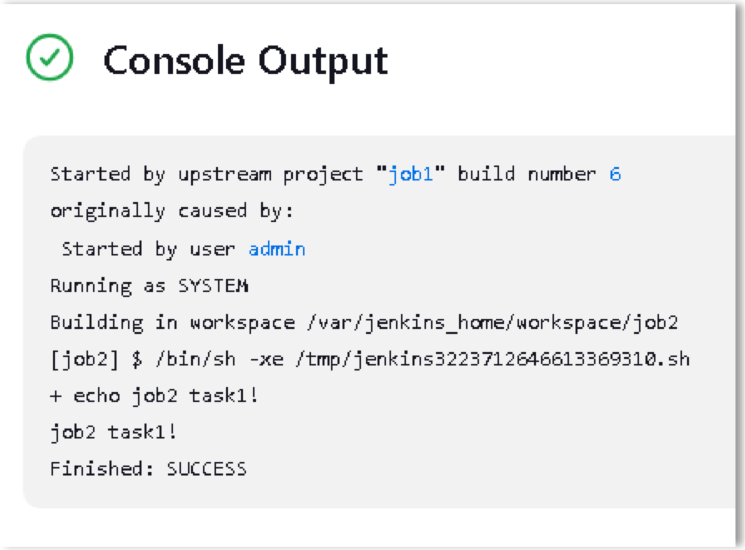


3- Configure Job1's post-build action to trigger building Job2

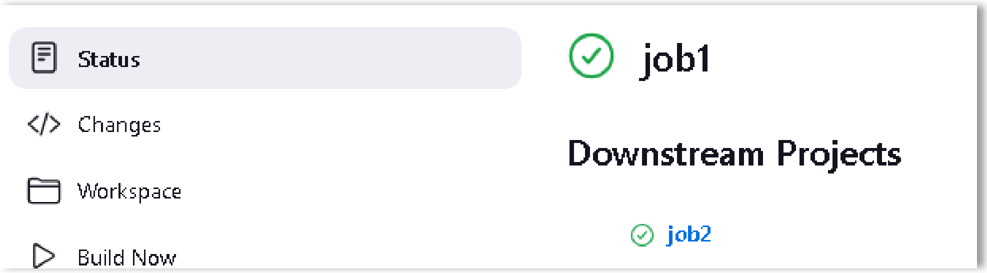


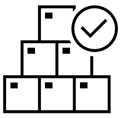
4- Build Job1 

and note that both jobs get built
View the console output of Job1 and Job2 for proof



5- View Job1's status for the Downstream project (Job2)





Congratulations, you have successfully created a Jenkins build chain.
Please continue to Lab2 to explore building and deploying a Maven app stored on GIT.